

1P-G2-IATI-resuelto.pdf



andreagb03



Inteligencia Artificial



3º Grado en Ingeniería Informática - Tecnologías Informáticas



Escuela Técnica Superior de Ingeniería Informática
Universidad de Sevilla



[Accede al documento original](#)

INTERCAMBIA y AHORRA en LA VUELTA A LA UNI

5 AÑOS DE GARANTÍA en todo

CeX
webuy.com



27/12/24, 12:17

1P G2 IATI resuelto

Inteligencia Artificial

Parte práctica de la primera prueba de evaluación alternativa. Curso 2022-23

Grupo 2

```
In [13]: # Apellidos:
          # Nombre:
```

```
In [15]: import random
```

```
In [17]: class Problema_Genetico(object):
          def __init__(self, genes, longitud_individuos, decodifica, fitness, opt):
              self.genes = genes
              self.longitud_individuos = longitud_individuos
              self.decodifica = decodifica
              self.fitness = fitness
              self.opt = opt
```

Ejemplo de Problema Genético. A continuación se presenta (usando la función `binario_a_decimal` que también se incluye) la definición de un objeto `cuad_gen` que define la representación genética usada en el problema de encontrar el número entero entre 0 y 2^{10} que minimiza la función $f(x) = (x + 5)^2$.

```
In [20]: def binario_a_decimal(l):
          return sum(x*(2**i) for (i,x) in enumerate(l))

          cuad_gen=Problema_Genetico([0,1],
                                     10,
                                     binario_a_decimal,
                                     lambda x: (binario_a_decimal(x)+5)**2,
                                     min)
```

Nota: En los siguientes ejercicios, el estudiante puede modificar y añadir el código que considere necesario, más allá de completar los huecos indicados para ello.

Ejercicio 1 (0.5 puntos)

Vamos a definir las funciones auxiliares que serán necesarias para la implementación de nuestro algoritmo genético. Comenzaremos por la función

`poblacion_inicial(problema_genetico, N)`, que recibe una instancia de la clase `Problema_Genetico` y un número natural N y devuelve una lista con una población inicial aleatoria de N individuos. Cada individuo será una lista de genes elegidos aleatoriamente. Completa las partes que faltan en la función:

```
In [25]: # Solución:

          def poblacion_inicial(problema_genetico, N):
```



Escanéame

```

"""Recibe una instancia de la clase Problema_Genetico y un número natural N,
y devuelve una lista con una poblacion inicial aleatoria de N individuos
return [[random.choice(problema_genetico.genes)
        for _ in range(problema_genetico.longitud_individuos)]
        for _ in range(N)]

```

Ejemplo:

In [28]: poblacion_inicial(cuad_gen,6)

```

# Salida esperada (los valores binarios pueden variar):
#[[1, 1, 1, 1, 0, 0, 0, 0, 1, 0],
# [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
# [0, 1, 0, 0, 1, 0, 1, 0, 0, 0],
# [0, 1, 0, 1, 1, 0, 0, 0, 1, 0],
# [0, 1, 0, 0, 1, 1, 0, 1, 1, 1],
# [1, 1, 1, 0, 0, 1, 1, 1, 0, 0]]

```

Out[28]:

```

[[0, 0, 1, 1, 1, 1, 1, 1, 1, 0],
 [0, 1, 0, 1, 0, 0, 0, 0, 1, 1],
 [0, 0, 0, 1, 0, 0, 1, 0, 1, 1],
 [0, 1, 0, 0, 0, 1, 1, 0, 1, 0],
 [1, 0, 0, 0, 0, 0, 0, 1, 1, 1],
 [1, 1, 0, 0, 1, 0, 1, 0, 0, 1]]

```

Ejercicio 2 (0.75 puntos)

A continuación vamos a definir la función `cruza_padres(padres)`, que dada una lista de individuos "padres" devuelve un conjunto del mismo tamaño formado por el resultado de aplicar la función cruce en un punto, de dos en dos, y en el orden en el que aparecen. Lo haremos definiendo una función auxiliar `cruza(c1,c2)` que recibiendo dos individuos `c1` y `c2` devolverá una lista con dos hijos (`hijo1` e `hijo2`) que serán el resultado de aplicar el cruce en un punto a `c1` y `c2`. Completa las partes que faltan:

In [32]: # Solución:

```

def cruza(c1,c2):

    """ Cruce de dos individuos c1 y c2 """

    pos= random.randint(1,len(c1)) #=???? # posición de cruce, aleatoria, entre
    hijo1= c1[:pos] + c2[pos:] #=???? #cruce de la primera posición del c1 con
    hijo2= c2[:pos] + c1[pos:] #=???? #cruce de la primera posición del c2 con
    return [hijo1,hijo2]

def cruza_padres(padres):

    """Dado una población de padres, obtiene el resultado de cruzar de dos en dos
    en el orden en el que aparecen"""

    hijos=[]
    for j in range(0,len(padres),2): #for j in ???
        hijos.extend(cruza(padres[j],padres[j+1]))
    return hijos

```

Ejemplo:

```
In [35]: pob = poblacion_inicial(cuad_gen,6)
cruza(pob[1],pob[2])

# Salida esperada (los valores binarios pueden variar, debido a la aleatoriedad)
#[[0, 1, 0, 1, 1, 0, 0, 1, 0, 0], [1, 0, 0, 1, 0, 1, 1, 0, 0, 1]]
```

```
Out[35]: [[0, 0, 1, 1, 1, 0, 0, 1, 1, 0], [0, 1, 1, 0, 0, 0, 0, 0, 0, 0]]
```

```
In [37]: cruza_padres(poblacion_inicial(cuad_gen,6))

# Salida esperada (los valores binarios pueden variar):
#[[1, 1, 1, 1, 0, 0, 0, 0, 1, 0],
# [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
# [0, 1, 0, 0, 1, 0, 1, 0, 0, 0],
# [0, 1, 0, 1, 1, 0, 0, 0, 1, 0],
# [0, 1, 0, 0, 1, 1, 0, 1, 1, 1],
# [0, 0, 0, 1, 1, 0, 0, 0, 1, 1]]
```

```
Out[37]: [[0, 1, 1, 0, 0, 0, 1, 0, 1, 1],
[0, 1, 0, 0, 1, 1, 1, 0, 0, 1],
[1, 1, 1, 1, 1, 1, 0, 0, 0, 1],
[1, 1, 0, 0, 0, 1, 1, 1, 0, 0],
[0, 1, 1, 0, 1, 0, 1, 0, 0, 1],
[1, 1, 1, 1, 1, 0, 0, 1, 1, 1]]
```

Ejercicio 3 (0.5 puntos)

Definiremos ahora la función `muta_individuos(poblacion, prob, genes)` que obtiene el resultado de mutar cada individuo de la población con probabilidad `prob` (utilizando el conjunto de genes posibles). Para ello, definiremos una función auxiliar `muta(c, prob, genes)` que a partir de un individuo `c`, obtiene un nuevo individuo resultado de mutar (con una probabilidad `prob`), cada gen de `c`. La variable `genes` contiene la lista de genes posibles. Completa las partes que faltan:



27/12/24, 12:17

1P G2 IATI resuelto

```
In [40]: # Solución:
def muta(c, prob, genes):
    """A partir de un individuo c, se obtiene un nuevo individuo resultado de
    mutar (con una probabilidad prob), cada gen de c. La variable genes contiene
    la lista de genes posibles. Nótese que si prob es bajo, entonces
    es probable que el individuo devuelto sea igual a c"""
    cm=list(c) # una copia
    for i in range(len(cm)):
        if random.uniform(0,1) < prob: #if ???
            cm[i] = random.choice(genes) #=???
    return cm

def muta_individuos(poblacion, prob, genes):
    """Obtiene el resultado de mutar cada individuo de población,
    con probabilidad prob"""
    res=[]
    for ind in poblacion:
        res.append(muta(ind, prob, genes)) #???
    return res
```

Ejemplo:

```
In [43]: muta(pob[1], 0.5, cuad_gen.genes)

# Salida esperada (los valores binarios pueden variar):
#[[0, 1, 0, 1, 1, 0, 0, 1, 0, 0]]
```

```
Out[43]: [[0, 1, 1, 0, 0, 1, 1, 0, 0, 0]]
```

```
In [45]: muta_individuos(poblacion_inicial(cuad_gen, 6), 0.5, cuad_gen.genes)

# Salida esperada (los valores binarios pueden variar):
#[[1, 1, 1, 1, 0, 0, 0, 0, 1, 0],
# [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
# [0, 1, 0, 0, 1, 0, 1, 0, 0, 0],
# [0, 1, 0, 1, 1, 0, 0, 0, 1, 0],
# [0, 1, 0, 0, 1, 1, 0, 1, 1, 1],
# [0, 0, 0, 1, 1, 0, 0, 0, 1, 1]]
```

```
Out[45]: [[1, 1, 0, 1, 1, 1, 1, 0, 1],
[0, 0, 1, 1, 1, 0, 1, 0, 1, 0],
[1, 1, 0, 0, 1, 0, 1, 0, 0, 0],
[1, 1, 0, 0, 0, 0, 0, 1, 1, 1],
[0, 0, 1, 1, 0, 0, 1, 0, 0, 1],
[0, 0, 0, 0, 1, 1, 1, 1, 0, 1]]
```

Ejercicio 4 (0.5 puntos)

Por último, definiremos funciones para la selección por torneo. La función `selecciona_n_torneo(problema_genetico, poblacion, k, n)` recibirá un `problema_genetico`, una `población`, y dos números naturales n y k , y selecciona n **individuos** de la población, mediante n torneos de tamaño k . Definiremos una función auxiliar `selecciona_un_torneo` que recibe un `problema_genetico`, una `población`, un número natural k , y selecciona **un individuo** de la población, mediante un torneo de tamaño k (consejo: usar `random.sample()`)



Escanéame

```
In [48]: # Solución:
def seleccion_un_torneo(problema_genetico,poblacion,k):
    """ Recibe un problema_genetico, una población y un número natural k
        y selecciona un individuo de la población,
        mediante un torneo de tamaño k"""
    participantes = random.sample(poblacion,k) #=???
    return problema_genetico.opt(participantes,key=problema_genetico.fitness) #?

def selecciona_n_torneo(problema_genetico, poblacion, k,n):
    """ Recibe un problema_genetico, una población y un número natural k
        y selecciona n individuos de la población,
        mediante n torneos de tamaño k"""
    return [seleccion_un_torneo(problema_genetico,poblacion,k) for _ in range(n)]
    #return [?????? for _ in range(????)]
```

Ejemplos:

```
In [51]: seleccion_un_torneo(cuad_gen,pob,2)

# Salida esperada (los valores binarios pueden variar):
#[[0, 1, 0, 1, 1, 0, 0, 1, 0, 0]
```

```
Out[51]: [0, 0, 0, 1, 1, 1, 1, 0, 0, 0]
```

```
In [53]: selecciona_n_torneo(cuad_gen,pob,4,2)

# Salida esperada (los valores binarios pueden variar):
#[[0, 0, 0, 0, 0, 1, 0, 0, 1, 0], [1, 1, 1, 1, 0, 1, 0, 0, 0, 0]]
```

```
Out[53]: [[0, 1, 1, 0, 0, 0, 0, 0, 0, 0], [1, 0, 1, 0, 1, 0, 0, 0, 0, 0]]
```

Ejercicio 5 (0.75 puntos)

A continuación se presenta la función `algoritmo_genetico` que implementa el algoritmo genético descrito en la diapositiva 45 del tema 2, haciendo uso de las funciones auxiliares que hemos definido previamente. Sólo queda por implementar la función `nueva_generacion`, que a partir de una generación obtiene la siguiente. Se pide implementar esa función.

```
In [56]: # Solución:
def nueva_generacion(problema_genetico,poblacion,n_padres,n_directos,prob_mutar,
    padres =
        selecciona_n_torneo(problema_genetico,poblacion,k_torneo,n_padres) #=???
    directos=
        selecciona_n_torneo(problema_genetico,poblacion,k_torneo,n_directos) #=?
    hijos = cruza_padres(padres) #=???

    return muta_individuos(hijos+directos,prob_mutar,problema_genetico.genos) #r
```

```
In [58]: def algoritmo_genetico(problema_genetico,ngen,N,prop_cruces,prob_mutar,k_torneo)
    """Algoritmo genético que se describe en el tema 2, diapositiva 47.
        Devuelve el mejor elemento de la última generación, junto con su valoraci
        Recibe como entrada los siguientes argumentos:
        - problema_genetico: un objeto de la clase Problema_Genetico, con la repr
        problema de optimización.
```

INTERCAMBIA y AHORRA en LA VUELTA A LA UNI



[webuy.com](https://www.webuy.com)

```

- ngen: número de generaciones
- N: número de individuos en cada generación
- prop_cruces: proporción de individuos de la población que se usan como
- prob_mutar: probabilidad de mutación de los genes
- k_torneo: número de individuos que intervienen en cada torneo
"""
poblacion= poblacion_inicial(problema_genetico,N)
n_padres=round(N*prop_cruces)
n_padres= (n_padres if n_padres%2==0 else n_padres-1)
n_directos= N-n_padres

for i in range(ngen):
    poblacion=
        nueva_generacion(problema_genetico,poblacion,n_padres,n_directos,pro
    mejor_cr=
        problema_genetico.opt(poblacion, key=lambda x:problema_genetico.fitness(x))
    mejor=problema_genetico.decodifica(mejor_cr)

return (mejor,problema_genetico.fitness(mejor_cr))

```

Ejemplo, con el problema `cuad_gen`, cuyo mínimo se alcanza en 0, con valor 25:

```
In [61]: algoritmo_genetico(cuad_gen,100,20,0.6,0.05,5)
```

```
Out[61]: (0, 25)
```

Ejercicio 6 (0.5 puntos)

A continuación, evaluaremos el comportamiento del algoritmo genético implementado, aplicándolo a tres problemas de la mochila de distinta complejidad. La representación genética del problema de la mochila está basada en la idea que se describe en las diapositivas 50 y 51 del tema 2. Completar la definición de las dos funciones auxiliares siguientes.

```

In [64]: # Solución:
def decodifica_mochila(cromosoma, n_objetos, pesos, capacidad):
    """ Dado un cromosoma (lista de 0s y 1s), cambia los 1s por 0s a partir de la
    posición del primer objeto que ya no cabe en la mochila, contando de izquierda
    a derecha """
    objetos=[0]*n_objetos
    peso_acum=0
    i=0
    while peso_acum < capacidad and i < n_objetos:
        if cromosoma[i] == 1:
            if peso_acum + pesos[i] <= capacidad:
                objetos[i] = 1 #??? #actualiza objetos
                peso_acum += pesos[i] #??? #actualiza peso_acum
            else:
                break
        i+=1
    return objetos

def fitness_mochila(cromosoma, n_objetos, pesos, capacidad, valores):
    """Fitness de un cromosoma en el problema de la mochila. Suma de los valores
    de los objetos que están dentro de la mochila. Como en la decodificación anterior,
    a partir del primer objeto que ya no cabe, se consideran que no están en la

```



27/12/24, 12:17

1P G2 IATI resuelto

```
independiente de que en la correspondiente posición haya un 1 o un 0""
valor_acum=0
peso_acum=0
i=0
while peso_acum < capacidad and i < n_objetos:
    if cromosoma[i] == 1:
        if peso_acum + pesos[i] <= capacidad:
            valor_acum += valores[i] #??? #actualiza valor_acum
            peso_acum += pesos[i] #??? #actualiza peso_acum
        else:
            break
    i+=1
return valor_acum
```

A continuación se ejecuta el algoritmo genético sobre tres problemas de la mochila, con objeto de poder probar la implementación. Para valorar la solución encontrada por el algoritmo, se proporciona la solución óptima:

```
In [67]: # Problema de La mochila 1:
# 10 objetos, peso máximo 165
pesos1 = [23,31,29,44,53,38,63,85,89,82]
valores1 = [92,57,49,68,60,43,67,84,87,72]

m1g =Problema_Genetico([0,1],
                        10,
                        lambda x: decodifica_mochila(x, 10, pesos1, 165),
                        lambda x: fitness_mochila(x, 10, pesos1, 165, valores1),
                        max)

# Solución óptima= [1,1,1,1,0,1,0,0,0,0], con valor 309
#

algoritmo_genetico(m1g,200,20,0.8,0.05,3)
```

Out[67]: ([1, 1, 1, 1, 0, 1, 0, 0, 0, 0], 309)

```
In [69]: # Problema de La mochila 2:
# 15 objetos, peso máximo 750

pesos2 = [70,73,77,80,82,87,90,94,98,106,110,113,115,118,120]
valores2 = [135,139,149,150,156,163,173,184,192,201,210,214,221,229,240]

m2g = Problema_Genetico([0,1],
                        15,
                        lambda x: decodifica_mochila(x, 15, pesos2, 750),
                        lambda x: fitness_mochila(x, 15, pesos2, 750, valores2),
                        max)

# Solución óptima= [1,0,1,0,1,0,1,1,1,0,0,0,0,1,1] con valor 1458

algoritmo_genetico(m2g,1000,200,0.8,0.05,5)
```

Out[69]: ([0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1], 1454)



Escanéame

```

In [70]: # Problema de La mochila 3:
# 24 objetos, peso máximo 6404180

pesos3 = [382745,799601,909247,729069,467902, 44328,
          34610,698150,823460,903959,853665,551830,610856,
          670702,488960,951111,323046,446298,931161, 31385,496951,264724,224916,169
valores3 = [825594,1677009,1676628,1523970, 943972, 97426,
            69666,1296457,1679693,1902996,
            1844992,1049289,1252836,1319836, 953277,2067538, 675367,
            853655,1826027, 65731, 901489, 577243, 466257, 369261]

m3g = Problema_Genetico([0,1],
                        24,
                        lambda x: decodifica_mochila(x, 24, pesos3, 6404180),
                        lambda x: fitness_mochila(x, 24, pesos3, 6404180, valores3,
max))

# Solución óptima= [1,1,0,1,1,1,0,0,0,1,1,0,1,0,0,1,0,0,0,0,0,1,1,1] con valorac

algoritmo_genetico(m3g,1000,2000,0.8,0.05,5)

```

```

Out[70]: ([1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1],
          13549094)

```

```

In [ ]:

```